

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 import java.util.Scanner;
2 import java.util.InputMismatchException;
3 public class ATM{
4     public static void main(String[] args){
5         Scanner sc=new Scanner(System.in);
6         try{
7             System.out.println("Enter account balance: ");
8             int balance=sc.nextInt();
9             System.out.println("Enter withdrawal amount: ");
10            int amount=sc.nextInt();
11            if(balance<0){
12                throw new IllegalArgumentException("Invalid account bal
13            }
14            if(amount==0){
15                throw new ArithmeticException("Withdrawal amount cannot
16            }
17        }
18        if(amount<0){
19            throw new IllegalArgumentException("withdrawal cannot b
20        }
21        if(amount>balance){
22            System.out.println("Insufficient balance. ");
23        }else{
24            int r=balance-amount;
25            System.out.println("Remaining balance: "+r);
```

Test Input

10000 0

Output

Enter account balance:
Enter withdrawal amount:
Withdrawal amount cannot be 0.
Transaction completed.

🔴 Pending manual review


```

15         throw new ArithmeticException("Withdrawal amount cannot be negative.");
16     }
17     }
18     if (amount < 0) {
19         throw new IllegalArgumentException("withdrawal cannot be negative.");
20     }
21     if (amount > balance) {
22         System.out.println("Insufficient balance. ");
23     } else {
24         int r = balance - amount;
25         System.out.println("Remaining balance: " + r);
26     }
27 }
28 catch (IllegalArgumentException e) {
29     System.out.println(e.getMessage());
30 } catch (ArithmeticException e) {
31     System.out.println(e.getMessage());
32 }
33 finally {
34     System.out.println("Transaction completed. ");
35 }
36 }
37 }

```

Submitted

Your Fixed Code

```
1 import java.util.Scanner;
2 class PatientRegistration{
3     public static void main(String arg[]){
4         Scanner sc=new Scanner(System.in);
5         try{
6             System.out.println("Enter Patient name: ");
7             String inputname=sc.nextLine();
8             String name;
9             if(inputname.equalsIgnoreCase("NULL")||inputname.trim().isEmpty()){
10                 name=null;
11                 throw new NullPointerException("Name cannot be null");
12             }
13             else{
14                 name=inputname;
15             }
16             int age=sc.nextInt();
17             if(age<=18){
18                 throw new IllegalArgumentException("Error: Age should be greater than 18");
19             }
20             System.out.println("Patient name: "+name.toUpperCase());
21             System.out.println("Patient age : "+age);
22         }catch(IllegalArgumentException e){
```

Test Input

19

Output

```
Enter Patient name:
Name cannot be null
registration completed
```

 Pending manual review


```

11         throw new NullPointerException("Name cannot be null");
12     }
13     else{
14         name=inputname;
15     }
16     int age=sc.nextInt();
17     if(age<=18){
18         throw new IllegalArgumentException("Error: Age should
19     }
20     System.out.println("Patient name: "+name.toUpperCase());
21     System.out.println("Patient age : "+age);
22 } catch (IllegalArgumentException e) {
23     System.out.println(e.getMessage());
24 }
25 catch (NullPointerException e) {
26     System.out.println(e.getMessage());
27 }
28 finally{
29     System.out.println("registration completed");
30 }
31 }
32 }

```

Submitted

Your Fixed Code

```
1 import java.util.Scanner;
2 import java.util.InputMismatchException;
3 class ProductSearch{
4     public static void main(String args[]){
5         Scanner sc=new Scanner(System.in);
6         String products[]{"laptop","Mobile","Tablet","Headphone","keyb
7         try{
8             System.out.println("Enter product number[1-5] : ");
9             int pn=sc.nextInt();
10            if(pn<1||pn>5){
11                throw new IllegalArgumentException("Invalid product num
12            }
13            System.out.println("Selected product : "+products[pn-1]);
14        }catch(InputMismatchException e){
15            System.out.println("Please enter the number. ");
16        }catch(IllegalArgumentException e){
17            System.out.println(e.getMessage());
18        }finally{
19            System.out.println("Search completed. ");
20        }
21    }
22 }
```

Submitted

Test Input

5

Output

```
Enter product number[1-5] :
Selected product : keyboard
Search completed.
```

Pending manual review

Your Fixed Code

```
1 class Main{
2     public static void main(String args[]){
3         Reservation res=new Reservation();
4         Customer c1=new Customer(res,"Customer 1");
5         Customer c2=new Customer(res,"Customer 2");
6         c1.start();
7         c2.start();
8     }
9 }
10 class Reservation{
11     int seats= 1;
12     synchronized void bookseat(String customer){
13         if(seats>0){
14             System.out.println(customer+" is booking seat");
15             try{
16                 Thread.sleep(100);
17             }catch(InterruptedException e){
18                 System.out.println("Thread Interrupted");
19             }
20             seats--;
21             System.out.println(customer+" seat booked successfully");
22         }else{
23             System.out.println("No seat available.");
24         }
25     }
26 }
27 class Customer extends Thread{
```

Test Input

stdin input (optional)

Output

```
Customer 1 is booking seat
Customer 1 seat booked successfully
No seat available.
```

⌚ Pending manual review


```
25 }  
26 }  
27 class Customer extends Thread{  
28     Reservation reservation;  
29     String customerName;  
30  
31     Customer(Reservation reservation,String customerName){  
32         this.reservation=reservation;  
33         this.customerName=customerName;  
34     }  
35     public void run(){  
36         reservation.bookseat(customerName);  
37     }  
38 }
```



Submitted

Secure Exit

SIMATS Online Programming Lab

JAVA PROGRAMMING



Mohammed Subahaan H
192572165RD

Your Fixed Code

```
1 class Main{
2     public static void main(String args[]) throws InterruptedException{
3         BankAccount acc= new BankAccount();
4         ATM atm1=new ATM(acc,7000);
5         ATM atm2=new ATM(acc,6000);
6         atm1.setName("ATM 1");
7         atm2.setName("ATM 2");
8         atm1.start();
9         atm2.start();
10        atm1.join();
11        atm2.join();
12        System.out.println("Final Balance: "+acc.balance);
13    }
14 }
15 class BankAccount{
16     int balance=10000;
17     synchronized void withdraw(int amount){
18         if(balance>=amount){
19             System.out.println(Thread.currentThread().getName()+" withd
20             try{
21                 Thread.sleep(100);
22             }catch(InterruptedException e){
23                 System.out.println("Interrupted");
24                 Thread.currentThread().interrupt();
25                 return;
26             }
27             balance=balance-amount;
28             System.out.println(Thread.currentThread().getName()+" withd
29             false;
```

Test Input

stdin input (optional)

Output

```
ATM 1 withdrawing 7000
ATM 1 withdraw successfull
ATM 2 insufficient balance
Final Balance: 3000
```

Pending manual review


```

22     } catch (InterruptedException e) {
23         System.out.println("Interrupted");
24         Thread.currentThread().interrupt();
25         return;
26     }
27     balance=balance-amount;
28     System.out.println(Thread.currentThread().getName()+" with
29 }else{
30     System.out.println(Thread.currentThread().getName()+" insu
31 }
32 }
33 }
34 class ATM extends Thread{
35     BankAccount account;
36     int amount;
37     ATM(BankAccount account,int amount){
38         this.account=account;
39         this.amount=amount;
40     }
41     public void run(){
42         account.withdraw(amount);
43     }
44 }

```

Submitted